AI & Soft Computing Lab: Codes and Viva Notes

1. 8-Puzzle using a Heuristic (C++)

The 8-puzzle is a 3 x 3 board with tiles 1 to 8 and one blank (0). A move slides a tile into the blank (blank moves up, down, left or right). The goal is 1 2 3 / 4 5 6 / 7 8 0. Trying every move order is too slow, so a **heuristic** h(n) estimates how far a board is from the goal, and we always pick the move with the smallest h.

Heuristics

- **Misplaced tiles:** h(n) = number of tiles not in their goal position (blank not counted).
- **Manhattan distance (used here):** h(n) = sum over tiles of |row - goal row| + |col - goal col|. For tile t, goal row = (t-1)/3 and goal column = (t-1)%3.
- Manhattan is better because it measures how far each tile must travel, not just whether it is wrong. Both are **admissible** (never overestimate the real number of moves).

Method and key terms

- The code uses **greedy best-first search (hill climbing)**: try all moves, keep the one with the smallest h, repeat until h = 0. It can get stuck in a **local minimum** (no move lowers h), so there is a 50-step limit.
- **A*** instead uses **f(n) = g(n) + h(n)**, where g = moves made so far, and keeps OPEN/CLOSED lists. With an admissible h it always finds the shortest solution. Complexity $O(b^d)$.
- **Solvability:** count inversions (a bigger tile before a smaller one, blank ignored). Solvable only if the count is even.
- Run: `g++ puzzle.cpp -o puzzle` then `./puzzle`

Code

```
#include <iostream>
#include <cmath>
#include <algorithm>
using namespace std;

// Manhattan distance: sum of row + column distance of each tile from its goal place
int heuristic(int a[3][3]) {
    int h = 0;
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++) {
            if (a[i][j] == 0) continue;     // skip the blank
            int x = (a[i][j] - 1) / 3;      // goal row of this tile
            int y = (a[i][j] - 1) % 3;      // goal column of this tile
            h += abs(i - x) + abs(j - y);
        }
    return h;
}

// solvable if the number of inversions is even
bool solvable(int a[3][3]) {
    int arr[9], k = 0, inv = 0;
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            if (a[i][j] != 0)
                arr[k++] = a[i][j];
    for (int i = 0; i < 8; i++)
        for (int j = i + 1; j < 8; j++)
            if (arr[i] > arr[j])
                inv++;
    return inv % 2 == 0;
}
```

```
void display(int a[3][3]) {
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++)
            cout << a[i][j] << " ";
        cout << endl;
    }
    cout << endl;
}

int main() {
    int a[3][3];
    cout << "Enter puzzle:\n";
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            cin >> a[i][j];
    cout << "\n";

    if (!solvable(a)) {
        cout << "Puzzle is not solvable" << endl;
        return 0;
    }

    int steps = 0;
    while (heuristic(a) != 0 && steps < 50) {
        steps++;

        int x, y;                           // find the blank (0)
        for (int i = 0; i < 3; i++)
            for (int j = 0; j < 3; j++)
                if (a[i][j] == 0)
                    x = i, y = j;

        int dx[] = {-1, 1, 0, 0};           // up, down, left, right
        int dy[] = {0, 0, -1, 1};
        int best = 100, bx = x, by = y;

        for (int k = 0; k < 4; k++) {       // try all 4 moves
            int nx = x + dx[k];
            int ny = y + dy[k];
            if (nx >= 0 && nx < 3 && ny >= 0 && ny < 3) {
                swap(a[x][y], a[nx][ny]);   // try the move
                int h = heuristic(a);
                if (h < best) {             // remember the move with the smallest h
                    best = h;
                    bx = nx;
                    by = ny;
                }
                swap(a[x][y], a[nx][ny]);   // undo the move
            }
        }

        swap(a[x][y], a[bx][by]);           // make the best move
        display(a);
    }

    if (heuristic(a) == 0)
        cout << "Goal Reached in " << steps << " steps" << endl;
    else
        cout << "Stuck: no better move (local minimum)" << endl;
    return 0;
}
```

Sample run

```
Enter puzzle:
4 1 3
0 2 6
7 5 8

0 1 3
4 2 6
7 5 8

1 0 3
4 2 6
7 5 8

1 2 3
4 0 6
7 5 8

1 2 3
4 5 6
7 0 8

1 2 3
4 5 6
7 8 0

Goal Reached in 5 steps
```

2. McCulloch-Pitts Neuron

The M-P neuron is the earliest model of a neuron. It adds weighted inputs and fires if the sum reaches a threshold. Its weights and threshold are **fixed by the designer**; it does not learn (unlike the perceptron).

Formulas

- net = x1 w1 + x2 w2 + ... + xn wn
- output y = 1 if net >= theta, else 0
- Weight +1 is **excitatory** (pushes towards 1); weight -1 is **inhibitory** (pushes towards 0).
- Absolute inhibition: theta > n w - p (n = excitatory inputs, w = their weight, p = inhibitory weight size).

Finding theta in the code

- theta must be <= the smallest net of rows with y = 1, and > the largest net of rows with y = 0. So theta = min(net of 1-rows); weights are valid if max(net of 0-rows) < theta.

Weights to remember

- AND: 1, 1, theta 2. OR: 1, 1, theta 1. NAND: -1, -1, theta -1. NOR: -1, -1, theta 0. A AND NOT B: 1, -1, theta 1. NOT: -1, theta 0.

XOR and adders

- XOR is **not linearly separable**: its 1s (0,1) and (1,0) are diagonally opposite its 0s, so no single straight line separates them. It needs a hidden layer: z1 = x1 AND NOT x2, z2 = NOT x1 AND x2, y = z1 OR z2.
- Half adder: Sum = A XOR B, Carry = A AND B.
- Full adder: Sum = A XOR B XOR C, Carry = AB + (A XOR B)C (two half adders and an OR).

Single gates (change only the y line)

AND y = [0,0,0,1]; OR y = [0,1,1,1]; NAND y = [1,1,1,0]; NOR y = [1,0,0,0]; A AND NOT B y = [0,0,1,0].

```
x1 = [0, 0, 1, 1]
x2 = [0, 1, 0, 1]
y  = [1, 1, 1, 0]          # NAND (change this line for another gate)

def calc_net(w1, w2):
    net = []
    for i in range(4):
        net.append(x1[i] * w1 + x2[i] * w2)
    return net

def find_theta(net):
    on = []                 # net values where y = 1
    off = []                # net values where y = 0
    for i in range(4):
        if y[i] == 1:
            on.append(net[i])
        else:
            off.append(net[i])
    theta = min(on)
    if max(off) < theta:
        return theta        # valid
    else:
        return None         # not valid

it = 1
while True:
    print("\nIteration", it)
    w1 = int(input("Enter w1: "))
    w2 = int(input("Enter w2: "))
    net = calc_net(w1, w2)
    theta = find_theta(net)
```

```
    print("x1 x2 y | net")
    for i in range(4):
        print(x1[i], x2[i], y[i], " |", net[i])

    if theta is None:
        print("Not valid, try other weights")
        it += 1
    else:
        print("Valid! theta =", theta)
        break
```

Sample run

```
Iteration 1
Enter w1: 1
Enter w2: 1
x1 x2 y | net
0 0 1  | 0
0 1 1  | 1
1 0 1  | 1
1 1 0  | 2
Not valid, try other weights

Iteration 2
Enter w1: -1
Enter w2: -1
x1 x2 y | net
0 0 1  | 0
0 1 1  | -1
1 0 1  | -1
1 1 0  | -2
Valid! theta = -1
```

NOT gate

```
x1 = [0, 1]
y  = [1, 0]                # NOT

def calc_net(w1):
    net = []
    for i in range(2):
        net.append(x1[i] * w1)
    return net

def find_theta(net):
    on = []                # net values where y = 1
    off = []               # net values where y = 0
    for i in range(2):
        if y[i] == 1:
            on.append(net[i])
        else:
            off.append(net[i])
    theta = min(on)
    if max(off) < theta:
        return theta       # valid
    else:
        return None        # not valid
```

```
it = 1
while True:
    print("\nIteration", it)
    w1 = int(input("Enter w1: "))
    net = calc_net(w1)
    theta = find_theta(net)

    print("x1 y | net")
    for i in range(2):
        print(x1[i], y[i], " |", net[i])

    if theta is None:
        print("Not valid, try other weights")
        it += 1
    else:
        print("Valid! theta =", theta)
        break
```

Sample run

```
Iteration 1
Enter w1: 1
x1 y | net
0 1  | 0
1 0  | 1
Not valid, try other weights

Iteration 2
Enter w1: -1
x1 y | net
0 1  | 0
1 0  | -1
Valid! theta = 0
```

XOR (3 neurons, weights entered for each)

Type z1: 1, -1; z2: -1, 1; y: 1, 1. For XNOR use y = [1,0,0,1] and type -1, -1 for the y neuron.

```
x1 = [0, 0, 1, 1]
x2 = [0, 1, 0, 1]
y  = [0, 1, 1, 0]          # XOR
z1_target = [0, 0, 1, 0]   # z1 = x1 AND NOT x2
z2_target = [0, 1, 0, 0]   # z2 = NOT x1 AND x2

def calc_net(x1, x2, w1, w2):
    net = []
    for i in range(4):
        net.append(x1[i] * w1 + x2[i] * w2)
    return net
```

```
def find_theta(net, target):
    on = []
    off = []
    for i in range(4):
        if target[i] == 1:
            on.append(net[i])
        else:
            off.append(net[i])
    theta = min(on)
    if max(off) < theta:
        return theta
    else:
        return None

def make_neuron(name, x1, x2, target):
    print("\n----- Neuron", name, "-----")
    while True:
        w1 = int(input("Enter w1: "))
        w2 = int(input("Enter w2: "))
        net = calc_net(x1, x2, w1, w2)
        theta = find_theta(net, target)
        print("inputs | target | net")
        for i in range(4):
            print(x1[i], x2[i], "  |", target[i], "     |", net[i])
        if theta is None:
            print("Not valid, try other weights")
        else:
            print("Valid! theta =", theta)
            return target

z1 = make_neuron("z1 (x1 AND NOT x2)", x1, x2, z1_target)
z2 = make_neuron("z2 (NOT x1 AND x2)", x1, x2, z2_target)
out = make_neuron("y (z1 OR z2)", z1, z2, y)

print("\nXOR")
print("x1 x2 | z1 z2 | y")
for i in range(4):
    print(x1[i], x2[i], " |", z1[i], z2[i], " |", out[i])
```

Sample run

```
----- Neuron z1 (x1 AND NOT x2) -----
Enter w1: 1
Enter w2: -1
inputs | target | net
0 0   | 0      | 0
0 1   | 0      | -1
1 0   | 1      | 1
1 1   | 0      | 0
Valid! theta = 1

----- Neuron z2 (NOT x1 AND x2) -----
Enter w1: -1
Enter w2: 1
inputs | target | net
0 0   | 0      | 0
0 1   | 1      | 1
1 0   | 0      | -1
1 1   | 0      | 0
Valid! theta = 1

----- Neuron y (z1 OR z2) -----
Enter w1: 1
Enter w2: 1
inputs | target | net
0 0   | 0      | 0
0 1   | 1      | 1
1 0   | 1      | 1
0 0   | 0      | 0
Valid! theta = 1

XOR
x1 x2 | z1 z2 | y
0 0  | 0 0  | 0
0 1  | 0 1  | 1
1 0  | 1 0  | 1
1 1  | 0 0  | 0
```

Half adder

```python
def mp(a, b, w1, w2, theta):
    net = a * w1 + b * w2
    if net >= theta:
        return 1
    return 0

def AND(a, b):
    return mp(a, b, 1, 1, 2)

def OR(a, b):
    return mp(a, b, 1, 1, 1)

def XOR(a, b):
    z1 = mp(a, b, 1, -1, 1)      # a AND NOT b
    z2 = mp(a, b, -1, 1, 1)      # NOT a AND b
    return OR(z1, z2)
```

```
print("Half Adder")
print("A B | Sum Carry")
for a in [0, 1]:
    for b in [0, 1]:
        print(a, b, "|", XOR(a, b), "   ", AND(a, b))
```

Sample run

```
Half Adder
A B | Sum Carry
0 0 | 0    0
0 1 | 1    0
1 0 | 1    0
1 1 | 0    1
```

Full adder

```
def mp(a, b, w1, w2, theta):
    net = a * w1 + b * w2
    if net >= theta:
        return 1
    return 0

def AND(a, b):
    return mp(a, b, 1, 1, 2)

def OR(a, b):
    return mp(a, b, 1, 1, 1)

def XOR(a, b):
    z1 = mp(a, b, 1, -1, 1)      # a AND NOT b
    z2 = mp(a, b, -1, 1, 1)      # NOT a AND b
    return OR(z1, z2)

print("Full Adder")
print("A B C | Sum Carry")
for a in [0, 1]:
    for b in [0, 1]:
        for c in [0, 1]:
            s = XOR(XOR(a, b), c)                    # Sum = A XOR B XOR C
            carry = OR(AND(a, b), AND(XOR(a, b), c))  # Carry = AB + (A XOR B)C
            print(a, b, c, "|", s, "   ", carry)
```

Sample run

```
Full Adder
A B C | Sum Carry
0 0 0 | 0    0
0 0 1 | 1    0
0 1 0 | 1    0
0 1 1 | 0    1
1 0 0 | 1    0
1 0 1 | 0    1
1 1 0 | 0    1
1 1 1 | 1    1
```

3. Perceptron and Adaline

Both are single-layer networks that **learn** their weights from data, epoch by epoch (one epoch = one pass over all rows). Bipolar values (-1, 1) are used so every row can change the weights (a 0 input would give zero change).

Perceptron learning rule

- yin = b + x1 w1 + x2 w2
- y = 1 if yin > 0, 0 if yin = 0, -1 if yin < 0
- If y != t: dw = alpha t x, db = alpha t, w(new) = w(old) + dw. If y = t: no change.
- Stops when an epoch has no weight change. Start w = b = 0, alpha = 1.
- **Convergence theorem:** if the data is linearly separable, the perceptron finds correct weights in finite steps. It cannot learn XOR.

Adaline (delta rule / LMS)

- dw = alpha (t - yin) x, db = alpha (t - yin); updates on **every** row, using yin (before activation).
- Error = $(t - yin)^2$; total error falls each epoch but does not reach 0. Start w = b = 0.1, alpha = 0.1.

Difference

- Perceptron: uses y after activation, updates only when wrong. Adaline: uses yin, always updates, minimises squared error.
- M-P vs perceptron: M-P has fixed hand-set weights; perceptron learns them. Bias shifts the decision line away from the origin.
- Gates: AND t = [1,-1,-1,-1]; OR t = [1,1,1,-1]; NAND t = [-1,1,1,1]; NOR t = [-1,-1,-1,1] (change only the t line).

Perceptron (AND)

```
x1 = [1, 1, -1, -1]
x2 = [1, -1, 1, -1]
t  = [1, -1, -1, -1]        # AND (bipolar). Change this line for another gate

w1 = 0
w2 = 0
b = 0
alpha = 1

def activation(yin):
    if yin > 0:
        return 1
    elif yin == 0:
        return 0
    else:
        return -1
```

```
for epoch in range(1, 11):
    print("\nEpoch", epoch)
    print("x1 x2  t | yin  y | dw1 dw2 db | w1 w2 b")
    changed = False
    for i in range(4):
        yin = b + x1[i] * w1 + x2[i] * w2
        y = activation(yin)
        if y != t[i]:                          # wrong output -> update
            dw1 = alpha * t[i] * x1[i]
            dw2 = alpha * t[i] * x2[i]
            db = alpha * t[i]
            changed = True
        else:                                  # correct output -> no change
            dw1 = 0
            dw2 = 0
            db = 0
        w1 = w1 + dw1
        w2 = w2 + dw2
        b = b + db
        print(x1[i], x2[i], t[i], " |", yin, y, " |", dw1, dw2, db, " |", w1, w2, b)
    if not changed:
        print("No weight changed in this epoch, so training is complete.")
        break

print("\nFinal weights: w1 =", w1, " w2 =", w2, " b =", b)
print("Testing:")
for i in range(4):
    yin = b + x1[i] * w1 + x2[i] * w2
    print(x1[i], x2[i], "-> output", activation(yin), " target", t[i])
```

Sample run

```
Epoch 1
x1 x2  t | yin  y | dw1 dw2 db | w1 w2 b
1 1 1  | 0 0  | 1 1 1  | 1 1 1
1 -1 -1  | 1 1  | -1 1 -1  | 0 2 0
-1 1 -1  | 2 1  | 1 -1 -1  | 1 1 -1
-1 -1 -1  | -3 -1  | 0 0 0  | 1 1 -1

Epoch 2
x1 x2  t | yin  y | dw1 dw2 db | w1 w2 b
1 1 1  | 1 1  | 0 0 0  | 1 1 -1
1 -1 -1  | -1 -1  | 0 0 0  | 1 1 -1
-1 1 -1  | -1 -1  | 0 0 0  | 1 1 -1
-1 -1 -1  | -3 -1  | 0 0 0  | 1 1 -1
No weight changed in this epoch, so training is complete.

Final weights: w1 = 1  w2 = 1  b = -1
Testing:
1 1 -> output 1  target 1
1 -1 -> output -1  target -1
-1 1 -> output -1  target -1
-1 -1 -> output -1  target -1
```

Adaline (OR)

```
x1 = [1, 1, -1, -1]
x2 = [1, -1, 1, -1]
t  = [1, 1, 1, -1]          # OR (bipolar). Change this line for another gate

w1 = 0.1
w2 = 0.1
b = 0.1
alpha = 0.1

for epoch in range(1, 6):
    print("\nEpoch", epoch)
    print("x1 x2  t |  yin   t-yin |  w1     w2     b    | error")
    total = 0
    for i in range(4):
        yin = b + x1[i] * w1 + x2[i] * w2
        diff = t[i] - yin                   # t - yin
        w1 = w1 + alpha * diff * x1[i]      # delta rule: always update
        w2 = w2 + alpha * diff * x2[i]
        b = b + alpha * diff
        error = diff ** 2
        total = total + error
        print(x1[i], x2[i], t[i], " |", round(yin, 3), round(diff, 3), " |",
              round(w1, 3), round(w2, 3), round(b, 3), " |", round(error, 3))
    print("Total error in epoch", epoch, "=", round(total, 3))

print("\nFinal weights: w1 =", round(w1, 3), " w2 =", round(w2, 3), " b =", round(b, 3))
print("Testing:")
for i in range(4):
    yin = b + x1[i] * w1 + x2[i] * w2
    if yin >= 0:
        y = 1
    else:
        y = -1
    print(x1[i], x2[i], "-> output", y, " target", t[i])
```

Sample run

```
Epoch 1
x1 x2  t |  yin   t-yin |  w1     w2     b    | error
1 1 1  | 0.3 0.7  | 0.17 0.17 0.17  | 0.49
1 -1 1  | 0.17 0.83  | 0.253 0.087 0.253  | 0.689
-1 1 1  | 0.087 0.913  | 0.162 0.178 0.344  | 0.834
-1 -1 -1  | 0.004 -1.004  | 0.262 0.279 0.244  | 1.009
Total error in epoch 1 = 3.021

Epoch 2
x1 x2  t |  yin   t-yin |  w1     w2     b    | error
1 1 1  | 0.785 0.215  | 0.284 0.3 0.265  | 0.046
1 -1 1  | 0.249 0.751  | 0.359 0.225 0.341  | 0.564
-1 1 1  | 0.207 0.793  | 0.279 0.304 0.42  | 0.629
-1 -1 -1  | -0.164 -0.836  | 0.363 0.388 0.336  | 0.699
Total error in epoch 2 = 1.938

Epoch 3
x1 x2  t |  yin   t-yin |  w1     w2     b    | error
1 1 1  | 1.087 -0.087  | 0.354 0.379 0.328  | 0.008
1 -1 1  | 0.303 0.697  | 0.424 0.31 0.397  | 0.486
-1 1 1  | 0.283 0.717  | 0.352 0.381 0.469  | 0.514
-1 -1 -1  | -0.265 -0.735  | 0.426 0.455 0.395  | 0.541
Total error in epoch 3 = 1.549

Epoch 4
x1 x2  t |  yin   t-yin |  w1     w2     b    | error
1 1 1  | 1.276 -0.276  | 0.398 0.427 0.368  | 0.076
1 -1 1  | 0.339 0.661  | 0.464 0.361 0.434  | 0.437
-1 1 1  | 0.331 0.669  | 0.397 0.428 0.501  | 0.448
-1 -1 -1  | -0.325 -0.675  | 0.465 0.496 0.433  | 0.456
Total error in epoch 4 = 1.418

Epoch 5
x1 x2  t |  yin   t-yin |  w1     w2     b    | error
1 1 1  | 1.394 -0.394  | 0.426 0.456 0.394  | 0.155
1 -1 1  | 0.363 0.637  | 0.489 0.393 0.458  | 0.405
-1 1 1  | 0.361 0.639  | 0.425 0.456 0.522  | 0.408
-1 -1 -1  | -0.36 -0.64  | 0.489 0.52 0.458  | 0.409
Total error in epoch 5 = 1.378

Final weights: w1 = 0.489  w2 = 0.52  b = 0.458
Testing:
1 1 -> output 1  target 1
1 -1 -> output 1  target 1
-1 1 -> output 1  target 1
-1 -1 -> output -1  target -1
```

4. Kohonen SOM and LVQ

Kohonen Self-Organising Map (unsupervised)

- No targets: the network groups similar inputs into clusters by itself. Each output unit has a weight vector representing one cluster.
- Distance: D(j) = sum $(x_i - w_{ij})^2$ (squared Euclidean). The unit with the smallest D wins (**competitive learning, winner takes all**).
- Only the winner learns: w(new) = w(old) + alpha (x - w(old)).
- In a full SOM the winner's neighbours also update; here the radius is 0. alpha is usually reduced over time.

Learning Vector Quantization (supervised)

- Same structure, but each output unit has a class and each input has a target T.
- Winner's class = T: w(new) = w(old) + alpha (x - w(old)) (move closer).
- Winner's class != T: w(new) = w(old) - alpha (x - w(old)) (move away).
- A new input gets the class of its nearest weight vector. Here the first two vectors are used as initial weights.

Difference

- Kohonen: unsupervised, clustering. LVQ: supervised, classification, corrects wrong winners.
- Note: the manual's Kohonen part (b) says D(2) = 0.08, but $(0.2 - 0.6)^2 = 0.16$, so the true winner is Y1.

Kohonen SOM

```
# Each row of W is the weight vector of one cluster unit (Y1, Y2, ...)
W = [[0.3, 0.5],
     [0.2, 0.6],
     [0.1, 0.7],
     [0.8, 0.9],
     [0.4, 0.2]]
inputs = [[0.2, 0.4]]       # input vectors to train on
alpha = 0.2
epochs = 1

def distance(x, w):
    d = 0
    for i in range(len(x)):
        d += (x[i] - w[i]) ** 2          # D(j) = sum (xi - wij)^2
    return d

for epoch in range(1, epochs + 1):
    print("\n===== Epoch", epoch, "=====")
    for x in inputs:
        print("\nInput:", x)
        D = []
        for j in range(len(W)):
            D.append(distance(x, W[j]))
            print("D(" + str(j + 1) + ") =", round(D[j], 4))

        J = D.index(min(D))              # winner = smallest distance
        print("Winner: Y" + str(J + 1))

        for i in range(len(x)):          # only the winner learns
            W[J][i] = W[J][i] + alpha * (x[i] - W[J][i])
        print("Updated weights of Y" + str(J + 1) + ":", [round(v, 4) for v in W[J]])

print("\nFinal weight matrix:")
for j in range(len(W)):
    print("Y" + str(j + 1) + ":", [round(v, 4) for v in W[j]])
```

Sample run

```
===== Epoch 1 =====

Input: [0.2, 0.4]
D(1) = 0.02
D(2) = 0.04
D(3) = 0.1
D(4) = 0.61
D(5) = 0.08
Winner: Y1
Updated weights of Y1: [0.28, 0.48]

Final weight matrix:
Y1: [0.28, 0.48]
Y2: [0.2, 0.6]
Y3: [0.1, 0.7]
Y4: [0.8, 0.9]
Y5: [0.4, 0.2]
```

LVQ

Set epochs = 20 to classify all training inputs correctly.

```
# reference (weight) vectors and the class each one stands for
W = [[0, 0, 1, 1],
     [1, 0, 0, 0]]
W_class = [1, 2]

# training vectors and their target classes
inputs  = [[0, 0, 0, 1], [1, 1, 0, 0], [0, 1, 1, 0]]
targets = [2, 1, 1]
alpha = 0.1
epochs = 1

def distance(x, w):
    d = 0
    for i in range(len(x)):
        d += (x[i] - w[i]) ** 2          # D(j) = sum (xi - wij)^2
    return d

for epoch in range(1, epochs + 1):
    print("\n===== Epoch", epoch, "=====")
    for k in range(len(inputs)):
        x = inputs[k]
        T = targets[k]
        print("\nInput:", x, " Target class:", T)
        D = []
        for j in range(len(W)):
            D.append(distance(x, W[j]))
            print("D(" + str(j + 1) + ") =", round(D[j], 4))

        J = D.index(min(D))              # winner = smallest distance
        print("Winner: unit", J + 1, " (class", W_class[J], ")")
```

```
        for i in range(len(x)):
            if W_class[J] == T:          # correct class -> move closer
                W[J][i] = W[J][i] + alpha * (x[i] - W[J][i])
            else:                        # wrong class -> move away
                W[J][i] = W[J][i] - alpha * (x[i] - W[J][i])
        if W_class[J] == T:
            print("Class matches -> moved closer")
        else:
            print("Class does not match -> moved away")
        print("Updated weights of unit", J + 1, ":", [round(v, 4) for v in W[J]])

print("\nFinal reference vectors:")
for j in range(len(W)):
    print("Unit", j + 1, "(class", W_class[j], "):", [round(v, 4) for v in W[j]])

# testing: give each input the class of its nearest reference vector
print("\nTesting:")
for k in range(len(inputs)):
    D = [distance(inputs[k], W[j]) for j in range(len(W))]
    J = D.index(min(D))
    print(inputs[k], "-> class", W_class[J], " (target", targets[k], ")")
```

Sample run

```
===== Epoch 1 =====

Input: [0, 0, 0, 1]  Target class: 2
D(1) = 1
D(2) = 2
Winner: unit 1  (class 1 )
Class does not match -> moved away
Updated weights of unit 1 : [0.0, 0.0, 1.1, 1.0]

Input: [1, 1, 0, 0]  Target class: 1
D(1) = 4.21
D(2) = 1
Winner: unit 2  (class 2 )
Class does not match -> moved away
Updated weights of unit 2 : [1.0, -0.1, 0.0, 0.0]

Input: [0, 1, 1, 0]  Target class: 1
D(1) = 2.01
D(2) = 3.21
Winner: unit 1  (class 1 )
Class matches -> moved closer
Updated weights of unit 1 : [0.0, 0.1, 1.09, 0.9]

Final reference vectors:
Unit 1 (class 1 ): [0.0, 0.1, 1.09, 0.9]
Unit 2 (class 2 ): [1.0, -0.1, 0.0, 0.0]

Testing:
[0, 0, 0, 1] -> class 1  (target 2 )
[1, 1, 0, 0] -> class 2  (target 1 )
[0, 1, 1, 0] -> class 1  (target 1 )
```

5. Associative Memory (Auto, Hetero, BAM)

Associative memories store patterns and recall them, even from a noisy or incomplete input. Weights are made in one step with the **Hebb rule**; there are no epochs.

Formulas

- Hebb rule: W = sum s^T t, i.e. W[i][j] += s[i] x t[j] ("fire together, wire together").
- Recall: yin[j] = sum_i x[i] W[i][j], then apply the activation.

Types

- **Auto-associative:** stores a pattern with itself (s = t); recalls the same pattern. Handles 1 to 2 missing values (0) or 1 mistake; 2 mistakes make yin = 0 and recall fails.
- **Hetero-associative:** stores input to a different output pattern; recalls the paired output.
- **BAM (bidirectional):** recalls both ways: X to Y with yin = x W, and Y to X with xin = y W^T.

Viva points

- A missing value (0) adds nothing to yin, so the other values still give the right sign.
- Bipolar (-1, 1) stores patterns better than binary, because -1 still carries information while 0 adds nothing.

Auto-associative

```
x = [-1, 1, 1, 1]           # pattern to store (bipolar)
n = len(x)

# Hebb rule: W = x^T x  (W[i][j] = x[i] * x[j])
W = []
for i in range(n):
    row = []
    for j in range(n):
        row.append(x[i] * x[j])
    W.append(row)
print("Weight matrix W:")
for row in W:
    print(row)

def recall(test):
    yin = [0] * n
    for i in range(n):
        for j in range(n):
            yin[j] += test[i] * W[i][j]  # yin = x . W
    y = []
    for v in yin:
        if v > 0:
            y.append(1)
        else:
            y.append(-1)
    return yin, y
```

```
tests = [
    ["Same input", [-1, 1, 1, 1]],
    ["One missing", [0, 1, 1, 1]],
    ["One mistake", [-1, -1, 1, 1]],
    ["Two missing", [0, 0, 1, 1]],
    ["Two mistakes", [-1, -1, -1, 1]],
]
for name, t in tests:
    yin, y = recall(t)
    print("\n" + name, ": input", t)
    print("yin =", yin, " y =", y)
    if y == x:
        print("Pattern recognised")
    else:
        print("Pattern NOT recognised")
```

Sample run

```
Weight matrix W:
[1, -1, -1, -1]
[-1, 1, 1, 1]
[-1, 1, 1, 1]
[-1, 1, 1, 1]

Same input : input [-1, 1, 1, 1]
yin = [-4, 4, 4, 4]  y = [-1, 1, 1, 1]
Pattern recognised

One missing : input [0, 1, 1, 1]
yin = [-3, 3, 3, 3]  y = [-1, 1, 1, 1]
Pattern recognised

One mistake : input [-1, -1, 1, 1]
yin = [-2, 2, 2, 2]  y = [-1, 1, 1, 1]
Pattern recognised

Two missing : input [0, 0, 1, 1]
yin = [-2, 2, 2, 2]  y = [-1, 1, 1, 1]
Pattern recognised

Two mistakes : input [-1, -1, -1, 1]
yin = [0, 0, 0, 0]  y = [-1, -1, -1, -1]
Pattern NOT recognised
```

Hetero-associative

```
s = [[1, 0, 0, 0],          # input patterns
     [1, 1, 0, 0],
     [0, 0, 0, 1],
     [0, 0, 1, 1]]
t = [[0, 1],                # target patterns
     [0, 1],
     [1, 0],
     [1, 0]]
n = len(s[0])               # input units
m = len(t[0])               # output units
```

```
# Hebb rule: W = sum of s^T t over all pairs
W = []
for i in range(n):
    W.append([0] * m)
for p in range(len(s)):
    for i in range(n):
        for j in range(m):
            W[i][j] += s[p][i] * t[p][j]
print("Weight matrix W:")
for row in W:
    print(row)

def recall(x):
    yin = [0] * m
    for i in range(n):
        for j in range(m):
            yin[j] += x[i] * W[i][j]     # yin = x . W
    y = []
    for v in yin:
        if v > 0:
            y.append(1)
        else:
            y.append(0)
    return yin, y

for p in range(len(s)):
    yin, y = recall(s[p])
    print("\nInput", s[p], "-> yin =", yin, " y =", y, " target", t[p])
```

Sample run

```
Weight matrix W:
[0, 2]
[0, 1]
[1, 0]
[2, 0]

Input [1, 0, 0, 0] -> yin = [0, 2]  y = [0, 1]  target [0, 1]

Input [1, 1, 0, 0] -> yin = [0, 3]  y = [0, 1]  target [0, 1]

Input [0, 0, 0, 1] -> yin = [2, 0]  y = [1, 0]  target [1, 0]

Input [0, 0, 1, 1] -> yin = [3, 0]  y = [1, 0]  target [1, 0]
```

BAM (letters E and F)

```
# letters on a 5x3 grid: 1 = *, -1 = .
E = [1, 1, 1,
     1, -1, -1,
     1, 1, 1,
     1, -1, -1,
     1, 1, 1]
F = [1, 1, 1,
     1, 1, 1,
     1, -1, -1,
     1, -1, -1,
     1, -1, -1]
tE = [-1, 1]                # target for E
tF = [1, 1]                 # target for F
```

```
X = [E, F]
T = [tE, tF]
n = 15                      # X-layer units
m = 2                       # Y-layer units

# Hebb rule: W = E^T tE + F^T tF
W = []
for i in range(n):
    W.append([0] * m)
for p in range(2):
    for i in range(n):
        for j in range(m):
            W[i][j] += X[p][i] * T[p][j]
print("Weight matrix W (15 x 2):")
for row in W:
    print(row)

def activation(v):
    if v > 0:
        return 1
    elif v < 0:
        return -1
    else:
        return 0

def forward(x):                         # X -> Y : yin = x . W
    yin = []
    for j in range(m):
        total = 0
        for i in range(n):
            total += x[i] * W[i][j]
        yin.append(total)
    return yin, [activation(v) for v in yin]

def backward(y):                        # Y -> X : xin = y . W^T
    xin = []
    for i in range(n):
        total = 0
        for j in range(m):
            total += y[j] * W[i][j]
        xin.append(total)
    return xin, [activation(v) for v in xin]

def show(x):                            # print a 15-value pattern as a 5x3 letter
    for r in range(5):
        line = ""
        for c in range(3):
            if x[r * 3 + c] == 1:
                line += "* "
            else:
                line += ". "
        print(line)

for name, x, tgt in [["E", E, tE], ["F", F, tF]]:
    yin, y = forward(x)
    print("\nForward: letter", name, "-> yin =", yin, " y =", y, " target", tgt)
```

```
for name, y, letter in [["E", tE, E], ["F", tF, F]]:
    xin, x = backward(y)
    print("\nBackward: target", y, "-> recalled pattern:")
    show(x)
    if x == letter:
        print("Correctly recalled letter", name)
    else:
        print("Recall failed")
```

Sample run

```
Weight matrix W (15 x 2):
[0, 2]
[0, 2]
[0, 2]
[0, 2]
[2, 0]
[2, 0]
[0, 2]
[-2, 0]
[-2, 0]
[0, 2]
[0, -2]
[0, -2]
[0, 2]
[-2, 0]
[-2, 0]

Forward: letter E -> yin = [-12, 18]  y = [-1, 1]  target [-1, 1]

Forward: letter F -> yin = [12, 18]  y = [1, 1]  target [1, 1]

Backward: target [-1, 1] -> recalled pattern:
* * *
* . .
* * *
* . .
* * *
Correctly recalled letter E

Backward: target [1, 1] -> recalled pattern:
* * *
* * *
* . .
* . .
* . .
Correctly recalled letter F
```

6. Fuzzy Sets and Relations

A crisp set has membership 0 or 1. A **fuzzy set** gives each element a membership value from 0 to 1 (how much it belongs). Operations are done element by element.

Set operations

- Union A U B = max(A, B); Intersection A n B = min(A, B); Complement A' = 1 - A; Difference A | B = min(A, 1 - B).
- Algebraic sum = A + B - A B; Algebraic product = A B; Bounded sum = min(1, A + B); Bounded difference = max(0, A - B).
- Laws of excluded middle do **not** hold: A U A' != X and A n A' != empty (e.g. A = 0.5 gives max = 0.5). De Morgan's laws still hold.

Relations

- Cartesian product R = A x B: R(x, y) = min(A(x), B(y)).
- Max-min composition: (R o S)(x, z) = max over y of min(R(x, y), S(y, z)).
- Max-product composition: (R o S)(x, z) = max over y of R(x, y) S(y, z).
- Composition works like matrix multiplication with max in place of sum and min (or product) in place of multiply. R's columns must equal S's rows. Max-product is always <= max-min.

Set operations

```
X = [0.64, 0.645, 0.65, 0.655, 0.66]      # universe
A = [0, 0.75, 1, 0.5, 0]                  # membership in fuzzy set A
B = [0, 0.25, 0.75, 1, 0.5]               # membership in fuzzy set B

union = []
inter = []
compA = []
compB = []
diff = []
alg_sum = []
alg_prod = []
bnd_sum = []
bnd_diff = []
for i in range(len(X)):
    union.append(max(A[i], B[i]))                          # A U B = max
    inter.append(min(A[i], B[i]))                          # A n B = min
    compA.append(round(1 - A[i], 4))                       # A' = 1 - A
    compB.append(round(1 - B[i], 4))                       # B' = 1 - B
    diff.append(round(min(A[i], 1 - B[i]), 4))             # A | B = min(A, 1 - B)
    alg_sum.append(round(A[i] + B[i] - A[i] * B[i], 4))    # A + B - A.B
    alg_prod.append(round(A[i] * B[i], 4))                 # A . B
    bnd_sum.append(round(min(1, A[i] + B[i]), 4))          # min(1, A + B)
    bnd_diff.append(round(max(0, A[i] - B[i]), 4))         # max(0, A - B)

print("Universe X         :", X)
print("A                  :", A)
print("B                  :", B)
print("Union A U B        :", union)
print("Intersection A n B :", inter)
print("Complement A'      :", compA)
print("Complement B'      :", compB)
print("Difference A | B   :", diff)
print("Algebraic sum      :", alg_sum)
print("Algebraic product  :", alg_prod)
print("Bounded sum        :", bnd_sum)
print("Bounded difference :", bnd_diff)
```

Sample run

```
Universe X         : [0.64, 0.645, 0.65, 0.655, 0.66]
A                  : [0, 0.75, 1, 0.5, 0]
B                  : [0, 0.25, 0.75, 1, 0.5]
Union A U B        : [0, 0.75, 1, 1, 0.5]
Intersection A n B : [0, 0.25, 0.75, 0.5, 0]
Complement A'      : [1, 0.25, 0, 0.5, 1]
Complement B'      : [1, 0.75, 0.25, 0, 0.5]
Difference A | B   : [0, 0.75, 0.25, 0, 0]
Algebraic sum      : [0, 0.8125, 1.0, 1.0, 0.5]
Algebraic product  : [0, 0.1875, 0.75, 0.5, 0.0]
Bounded sum        : [0, 1, 1, 1, 0.5]
Bounded difference : [0, 0.5, 0.25, 0, 0]
```

Cartesian product and compositions

```
# ---------- Cartesian product ----------
A = [0.2, 0.5, 1]           # fuzzy set A on x1, x2, x3
B = [0.3, 0.9]              # fuzzy set B on y1, y2

R_cart = []
for i in range(len(A)):
    row = []
    for j in range(len(B)):
        row.append(min(A[i], B[j]))       # mu(x, y) = min(muA(x), muB(y))
    R_cart.append(row)
print("Cartesian product A x B:")
for row in R_cart:
    print(row)

# ---------- Composition ----------
R = [[0.6, 0.3],
     [0.2, 0.9]]            # relation from X to Y (2 x 2)
S = [[1, 0.5, 0.3],
     [0.8, 0.4, 0.7]]       # relation from Y to Z (2 x 3)

rows = len(R)               # size of X
mid = len(S)                # size of Y (shared)
cols = len(S[0])            # size of Z

maxmin = []
maxprod = []
for i in range(rows):
    row1 = []
    row2 = []
    for k in range(cols):
        mins = []
        prods = []
        for j in range(mid):
            mins.append(min(R[i][j], S[j][k]))
            prods.append(round(R[i][j] * S[j][k], 4))
        row1.append(max(mins))            # max of the mins
        row2.append(max(prods))           # max of the products
    maxmin.append(row1)
    maxprod.append(row2)
```

```
print("\nMax-min composition R o S:")
for row in maxmin:
    print(row)
print("\nMax-product composition R o S:")
for row in maxprod:
    print(row)
```

Sample run

```
Cartesian product A x B:
[0.2, 0.2]
[0.3, 0.5]
[0.3, 0.9]

Max-min composition R o S:
[0.6, 0.5, 0.3]
[0.8, 0.4, 0.7]

Max-product composition R o S:
[0.6, 0.3, 0.21]
[0.72, 0.36, 0.63]
```

7. Fuzzy Logic Controller (scikit-fuzzy)

A fuzzy controller uses human-style IF-THEN rules instead of an exact mathematical model. Parts: fuzzifier, knowledge base (membership functions), rule base, inference engine, defuzzifier.

Steps

- **Fuzzification:** crisp input to membership values (e.g. level 70 is 0.33 Medium and 0.25 High).
- **Rule evaluation:** IF A AND B THEN C; firing strength = min(mu A, mu B).
- **Aggregation:** cut each rule's output set at its strength, combine with max.
- **Defuzzification (centroid):** output = sum z mu(z) / sum mu(z).

Membership functions

- Triangle [a, b, c]: 0 at a, 1 at b, 0 at c. Edge sets use shoulders, e.g. [min, min, mid] and [mid, max, max].
- Quick rule for a range: mid = (min + max)/2, q1 = min + range/4, q3 = min + 3 range/4. Low = [min, min, mid], Medium = [q1, mid, q3], High = [mid, max, max]. Or use var.automf(names=[...]).
- Sets must overlap so the output changes smoothly.

For a new case study change only

- The variables and ranges (Antecedent = input, Consequent = output; np.arange(min, max + 1, 1)), the sets, the rules (3 x 3 = 9), and the sim.input / sim.output names.
- Mamdani (this controller) has fuzzy rule outputs and needs defuzzification; Sugeno (ANFIS) has formula outputs and uses a weighted average.

Water dam controller (template)

Graph lines at the end are optional; delete them if graphs are not asked.

```
import numpy as np
import skfuzzy as fuzz
from skfuzzy import control as ctrl

# ---------- 1. variables and their ranges (CHANGE) ----------
level = ctrl.Antecedent(np.arange(0, 101, 1), 'level')       # input 1: water level %
rate = ctrl.Antecedent(np.arange(-10, 11, 1), 'rate')        # input 2: rate of change
gate = ctrl.Consequent(np.arange(0, 101, 1), 'gate')         # output: gate opening %

# ---------- 2. membership functions [a, b, c] (CHANGE) ----------
level['Low'] = fuzz.trimf(level.universe, [0, 0, 40])
level['Medium'] = fuzz.trimf(level.universe, [20, 50, 80])
level['High'] = fuzz.trimf(level.universe, [60, 100, 100])

rate['Falling'] = fuzz.trimf(rate.universe, [-10, -10, 0])
rate['Stable'] = fuzz.trimf(rate.universe, [-5, 0, 5])
rate['Rising'] = fuzz.trimf(rate.universe, [0, 10, 10])

gate['Small'] = fuzz.trimf(gate.universe, [0, 0, 50])
gate['Medium'] = fuzz.trimf(gate.universe, [25, 50, 75])
gate['Large'] = fuzz.trimf(gate.universe, [50, 100, 100])

# ---------- 3. rules: IF input1 AND input2 THEN output (CHANGE) ----------
rules = [
    ctrl.Rule(level['Low'] & rate['Falling'], gate['Small']),
    ctrl.Rule(level['Low'] & rate['Stable'], gate['Small']),
    ctrl.Rule(level['Low'] & rate['Rising'], gate['Medium']),
    ctrl.Rule(level['Medium'] & rate['Falling'], gate['Small']),
    ctrl.Rule(level['Medium'] & rate['Stable'], gate['Medium']),
    ctrl.Rule(level['Medium'] & rate['Rising'], gate['Large']),
    ctrl.Rule(level['High'] & rate['Falling'], gate['Medium']),
    ctrl.Rule(level['High'] & rate['Stable'], gate['Large']),
    ctrl.Rule(level['High'] & rate['Rising'], gate['Large']),
]
```

```
# ---------- 4. build controller, give inputs, compute ----------
system = ctrl.ControlSystem(rules)
sim = ctrl.ControlSystemSimulation(system)

sim.input['level'] = float(input("Enter water level (0-100): "))
sim.input['rate'] = float(input("Enter rate of change (-10 to 10): "))
sim.compute()
print("Gate opening =", round(sim.output['gate'], 2), "%")

# ---------- optional graphs ----------
import matplotlib.pyplot as plt
level.view()
rate.view()
gate.view(sim=sim)          # output sets with the result shaded
plt.show()
```

Sample run

```
Enter water level (0-100): 70
Enter rate of change (-10 to 10): 5
Gate opening = 78.89 %
```